And now suppose that we want to fetch two JSON values with this function:

The problem with this code is that it is unnecessarily sequential: the fetch of the second URL will not be-gin until the first fetch is complete. If the second URL does not depend on the value obtained from the first URL, then we should probably try to fetch the two values at the same time. This is a case where the Promise-based nature of async functions shows. In order to await a set of concurrently executing async functions, we use Promise.all() just as we would if working with Promises directly:

13.3.4 Implementation Details

Finally, in order to understand how async functions work, it may help to think about what is going on under the hood.

Suppose you write an async function like this:

You can think about this as a Promise-returning function wrapped around the body of your original func-tion:

It is harder to express the await keyword in terms of a syntax transformation like this one. But think of the await keyword as a marker that breaks a function body up into separate, synchronous chunks. An ES2017 interpreter can break the function body up into a sequence of separate subfunctions, each of which gets passed to the then() method of the await -marked Promise that precedes it.

13.4 Asynchronous Iteration

We began this chapter with a discussion of callback- and event-based asynchrony, and when we intro-duced Promises, we noted that they were useful for single-shot asynchronous computations but were not suitable for use with sources of repetitive asynchronous events, such as setInterval(), the “click” event in a web browser, or the “data” event on a Node stream. Because single Promises do not work for se-quences of asynchronous events, we also cannot use regular async functions and the await statements for these things.

ES2018 provides a solution, however. Asynchronous iterators are like the iterators described in
Chapter 12, but they are Promise-based and are meant to be used with a new form of the for/of loop: for/await.

13.4.1 The for/await Loop

Node 12 makes its readable streams asynchronously iterable. This means you can read successive chunks of data from a stream with a for/await loop like this one:

Like a regular await expression, the for/await loop is Promise-based. Roughly speaking, the asynchro-nous iterator produces a Promise and the for/await loop waits for that Promise to fulfill, assigns the ful-fillment value to the loop variable, and runs the body of the loop. And then it starts over, getting another Promise from the iterator and waiting for that new Promise to fulfill.

Suppose you have an array of URLs:

You can call fetch() on each URL to get an array of Promises:

We saw earlier in the chapter that we could now use Promise.all() to wait for all the Promises in the array to be fulfilled. But suppose we want the results of the first fetch as soon as they become available and don’t want to wait for all the URLs to be fetched. (Of course, the first fetch might take longer than any of the others, so this is not necessarily faster than using Promise.all().) Arrays are iterable, so we can iterate through the array of promises with a regular for/of loop:

This example code uses a regular for/of loop with a regular iterator. But because this iterator returns Promises, we can also use the new for/await for slightly simpler code:

In this case, the for/await loop just builds the await call into the loop and makes our code slightly more compact, but the two examples do exactly the same thing. Importantly, both examples will only work if they are within functions declared async; a for/await loop is no different than a regular await expression in that way.

It is important to realize, however, that we’re using for/await with a regular iterator in this example. Things are more interesting with fully asynchronous iterators.

13.4.2 Asynchronous Iterators

Let’s review some terminology from Chapter 12. An iterable object is one that can be used with a for/of loop. It defines a method with the symbolic name Symbol.iterator. This method returns an iterator ob-ject. The iterator object has a next() method, which can be called repeatedly to obtain the values of the iterable object. The next() method of the iterator object returns iteration result objects. The iteration re-sult object has a value property and/or a done property.

Asynchronous iterators are quite similar to regular iterators, but there are two important differences. First, an asynchronously iterable object implements a method with the symbolic name Symbol.asyncIterator instead of Symbol.iterator. (As we saw earlier, for/await is compatible with regular iterable objects but it prefers asynchronously iterable objects, and tries the Symbol.asyncIterator method before it tries the Symbol.iterator method.) Second, the next() method of an asynchronous iterator returns a Promise that resolves to an iterator result object instead of returning an iterator result object directly.

NOTE

In the previous section, when we used for/await on a regular, synchronously iterable array of Promises, we were working with synchronous iterator result objects in which the value property was a Promise object but the done property was synchronous. True asynchronous iterators return Promises for iteration result objects, and both the value and the done properties are asynchronous. The difference is a subtle one: with asynchronous iterators, the choice about when iteration ends can be made asynchronously.

13.4.3 Asynchronous Generators

As we saw in Chapter 12, the easiest way to implement an iterator is often to use a generator. The same is true for asynchronous iterators, which we can implement with generator functions that we declare async. An async generator has the features of async functions and the features of generators: you can use await as you would in a regular async function, and you can use yield as you would in a regular generator. But values that you yield are automatically wrapped in Promises. Even the syntax for async generators is a combination: async function and function * combine into async function *. Here is an example that shows how you might use an async generator and a for/await loop to repetitively run code at fixed intervals using loop syntax instead of a setInterval() callback function:

13.4.4 Implementing Asynchronous Iterators

Instead of using async generators to implement asynchronous iterators, it is also possible to implement them directly by defining an object with a Symbol.asyncIterator() method that returns an object with a next() method that returns a Promise that resolves to an iterator result object. In the following code, we re-implement the clock() function from the preceding example so that it is not a generator and in-stead just returns an asynchronously iterable object. Notice that the next() method in this example does not explicitly return a Promise; instead, we just declare next() to be async:

}

Và bây giờ, giả sử chúng ta muốn lấy hai giá trị dạng JSON bằng hàm này:

let value1 = await getJSON(url1);
let value2 = await getJSON(url2);

Vấn đề với đoạn mã này là việc thực hiện các thao tác lấy dữ liệu từ các URL diễn ra một cách tuần tự, một cách không cần thiết. Việc lấy dữ liệu từ URL thứ hai sẽ không bắt đầu cho đến khi việc lấy dữ liệu từ URL thứ nhất hoàn tất. Nếu việc lấy dữ liệu từ URL thứ hai không phụ thuộc vào kết quả từ URL thứ nhất, thì chúng ta nên cố gắng lấy dữ liệu từ cả hai URL cùng lúc. Đây chính là trường hợp mà tính năng dựa trên Promise của các hàm async phát huy tác dụng. Để chờ đợi kết quả của các hàm async được thực hiện đồng thời, chúng ta sử dụng Promise.all(), tương tự như cách chúng ta sử dụng Promises trực tiếp.

let [value1, value2] = await Promise.all([getJSON(url1), getJSON(url2)]);

13.3.4 Chi tiết về việc triển khai

Cuối cùng, để hiểu rõ cách thức hoạt động của async, có lẽ sẽ hữu ích nếu chúng ta tìm hiểu xem những gì đang diễn ra bên trong hệ thống này.

Giả sử bạn viết một hàm async như thế này:

async function f(x) { /* body */ }

Bạn có thể coi đây như một hàm trả về giá trị “Promise”

, được tạo ra bằng cách bao bọc toàn bộ nội dung của
hàm gốc lại trong hàm này.

function f(x) {
return new Promise(function(resolve, reject) {
try {
resolve((function(x) { /* body */ })(x));
}
catch(e) {
reject(e);
}
});
}

Việc biểu diễn từ khóa await thông qua các quy tắc cú pháp như thế này khá khó khăn. Nhưng hãy coi từ khóa await như một dấu hiệu dùng để chia thân hàm thành các phần riêng biệt, hoạt động đồng bộ với nhau. Trình thông dịch ES2017 có thể chia thân hàm thành nhiều hàm con; mỗi hàm con này sẽ được truyền vào phương thức then() của đối tượng Promise được đánh dấu bằng await.

13.4 Lặp đi lặp lại một cách bất đồng bộ Chúng ta bắt đầu chương này bằng việc thảo luận về các phương thức xử lý các sự kiện diễn ra một cách bất

đồng bộ. Khi giới thiệu khái niệm Promises, chúng ta đã nhận thấy rằng chúng rất hữu ích để xử lý các phép tính diễn ra một lần duy nhất một cách bất đồng bộ. Tuy nhiên, Promises không phù hợp để xử lý các chuỗi sự kiện diễn ra liên tục một cách bất đồng bộ, chẳng hạn như sự kiện “click” trong trình duyệt web, hoặc sự kiện “data” trong luồng dữ liệu của Node.js. Vì vậy, chúng ta cũng không thể sử dụng các hàm thông thường hoặc các câu lệnh liên quan đến Promises để xử lý các trường hợp này.

Tuy nhiên, ES2018 đã cung cấp một giải pháp cho vấn đề này. Các bộ lặp bất đồng bộ tương tự như các bộ lặp được mô tả trong Chương 12. Tuy nhiên, chúng hoạt động dựa trên cơ chế Promise, và được thiết kếđểđược sử dụng cùng với kiểu vòng lặp for/await.

13.4.1 Vòng lặp for/await

Node 12 giúp các luồng dữ liệu có thểđược đọc một cách bất đồng bộ. Điều này có nghĩa là bạn có thểđọc từng phần dữ liệu trong luồng đó bằng cách sử dụng vòng lặp for/await. Ví dụ:

const fs = require("fs");

async function parseFile(filename) {
let stream = fs.createReadStream(filename, { encoding: "utf-8"});
for await (let chunk of stream) {
parseChunk(chunk); // Assume parseChunk() is defined elsewhere
}
}

Giống như các biểu thức await thông thường, vòng lặp for/await cũng được xây dựng dựa trên cơ chế Promise. Nói một cách đơn giản, bộ lặp bất đồng bộ này tạo ra một đối tượng Promise; vòng lặp for/await sẽ chờ cho đến khi đối tượng Promise đó được thực hiện hoàn tất. Sau đó, giá trị thu được từ việc thực hiện đối tượng Promise đó sẽ được gán cho biến dùng trong vòng lặp, và phần mã bên trong vòng lặp sẽ được thực thi. Quá trình này được lặp đi lặp lại: vòng lặp tiếp tục nhận một đối tượng Promise mới từ bộ lặp bất đồng bộ, và chờ cho đến khi đối tượng Promise đó được thực hiện hoàn tất.

Giả sử bạn có một mảng chứa các địa chỉ URL:

const urls = [url1, url2, url3];

Bạn có thể gọi fetch() tại mỗi địa chỉ URL để nhận được một mảng các giá trị kiểu Promise.

const promises = urls.map(url => fetch(url));

Trong phần trước, chúng ta đã biết rằng có thể sử dụng Promise.all() để chờ đợi cho đến khi tất cả các lệnh truy vấn trong mảng được thực hiện xong. Nhưng giả sử chúng ta muốn nhận kết quả của lệnh truy vấn đầu tiên ngay khi nó được thực hiện xong, mà không muốn chờ đợi cho đến khi tất cả các lệnh truy vấn khác cũng được thực hiện xong. (Tất nhiên, lệnh truy vấn đầu tiên có thể mất nhiều thời gian hơn các lệnh truy vấn khác; vì vậy, cách này không chắc đã nhanh hơn việc sử dụng Promise.all().) Vì mảng là cấu trúc có thểđược duyệt từng phần một, nên chúng ta có thể duyệt qua các lệnh truy vấn trong mảng bằng cách sử dụng vòng lặp for/of.

for(const promise of promises) {
response = await promise;
handle(response);
}

Đoạn mã ví dụ này sử dụng vòng lặp for/of cùng với bộ lặp thông thường. Tuy nhiên, vì bộ lặp này trả về các giá trị kiểu Promise, chúng ta cũng có thể sử dụng cách viết for/await để tạo ra đoạn mã đơn giản hơn một

chút.

for await (const response of promises) {
handle(response);
}

Trong trường hợp này, vòng lặp for/await chỉ đơn giản là kết hợp lệnh gọi await vào bên trong vòng lặp mà thôi. Điều này giúp mã nguồn trở nên gọn gàng hơn một chút. Tuy nhiên, cả hai ví dụ trên đều thực hiện cùng một chức năng. Điều quan trọng là cả hai ví dụ chỉ hoạt động được nếu chúng nằm bên trong các hàm được khai báo bằng async. Về mặt này, vòng lặp for/await không khác gì một biểu thức await thông thường cả.

Tuy nhiên, cần lưu ý rằng trong ví dụ này, chúng ta đang sử dụng for/await cùng với các bộ lặp thông thường. Mọi thứ sẽ trở nên thú vị hơn khi sử dụng các bộ lặp phi đồng bộ.

13.4.2 Các bộ lặp bất đồng bộ

Hãy cùng ôn lại một số thuật ngữ trong Chương 12. Một đối tượng có thểđược lặp qua được là đối tượng có thể được sử dụng trong vòng lặp for/of. Đối tượng này có một phương thức mang tên ký hiệu Symbol.iterator. Phương thức này trả về một đối tượng iterator. Đối tượng iterator có phương thức next() , và phương thức này có thểđược gọi nhiều lần để lấy các giá trị của đối tượng có thểđược lặp qua. Phương thức next() của đối tượng iterator trả về các đối tượng chứa kết quả của quá trình lặp. Đối tượng chứa kết quả này có thuộc tính value và/hoặc thuộc tính done.

Các bộ lặp kiểu bất đồng bộ có nhiều điểm tương đồng với các bộ lặp thông thường. Tuy nhiên, có hai điểm khác biệt quan trọng. Thứ nhất, đối tượng có khả năng được lặp kiểu bất đồng bộ sẽ triển khai phương thức có tên ký hiệu là Symbol.asyncIterator, thay vì Symbol.iterator. (Như chúng ta đã biết, for/await tương thích với các đối tượng có khả năng được lặp kiểu thông thường; tuy nhiên, for/await ưu tiên sử dụng các đối tượng có khả năng được lặp kiểu bất đồng bộ. for/await sẽ cố gắng gọi phương thức Symbol.asyncIterator trước khi gọi phương thức Symbol.iterator.) Thứ hai, phương thức next() của các bộ lặp kiểu bất đồng bộ sẽ trả về một đối tượng loại Promise; đối tượng này sau đó mới được chuyển đổi thành kết quả của việc lặp, thay vì trực tiếp trả về kết quả ngay lập tức.

GHI CHÚ

Trong phần trước, khi chúng ta sử dụng for/await với một mảng các đối tượng Promise được xử lý một cách đồng bộ, chúng ta thực chất đang làm việc với các đối tượng kết quả của quá trình lặp. Trong các đối tượng này, thuộc tính value là đối tượng Promise, còn thuộc tính done thì mang tính chất đồng bộ. Ngược lại, các bộ lặp bất đồng bộ sẽ trả về các đối tượng Promise làm kết quả của quá trình lặp; cả hai thuộc tính value và done đều mang tính chất bất đồng bộ. Sự khác biệt này khá tinh tế: với các bộ lặp bất đồng bộ, việc quyết định thời điểm kết thúc quá trình lặp có thểđược thực hiện một cách bất đồng bộ.

13.4.3 Máy phát điện bất đồng bộ

Như chúng ta đã thấy trong Chương 12, cách dễ nhất để triển khai một trình duyệt (iterator) thường là sử dụng một hàm tạo (generator). Điều này cũng đúng với các trình duyệt không đồng bộ (asynchronous iterators), mà chúng ta có thể triển khai bằng các hàm tạo (generator functions) mà chúng ta khai báo với `

async

`

. Một hàm
tạo không đồng bộ (async generator) có các tính năng của các hàm không đồng bộ và các tính năng của hàm tạo: bạn có thể sử dụng

`

await `

như trong một hàm không đồng bộ thông thường, và bạn có thể sử dụng

`

yield `

như trong một hàm tạo thông thường. Tuy nhiên, các giá trị mà bạn

`

yield `

sẽ tự động được bọc
trong Promises. Ngay cả cú pháp cho các hàm tạo không đồng bộ cũng là một sự kết hợp:

`

async function

`

và ` function * ` kết hợp thành `

async function * `

. Dưới đây là một ví dụ cho thấy cách bạn có thể sử
dụng một hàm tạo không đồng bộ và một vòng lặp (`loop

`) để lặp đi lặp lại một đoạn mã tại các khoảng thời gian cốđịnh bằng cú pháp vòng lặp thay vì một hàm

`

callback`

:

// A Promise-based wrapper around setTimeout() that we can use await with. // Returns a Promise that fulfills in the specified number of milliseconds function elapsedTime(ms) {
return new Promise(resolve => setTimeout(resolve, ms));
}

// An async generator function that increments a counter and yields it
// a specified (or infinite) number of times at a specified interval. async function* clock(interval, max=Infinity) {
for(let count = 1; count <= max; count++) { // regular for loop
await elapsedTime(interval); // wait for time to pass
yield count; // yield the counter
}
}

// A test function that uses the async generator with for/await
async function test() { // Async so we can use for/await
for await (let tick of clock(300, 100)) { // Loop 100 times every 300ms console.log(tick);
}
}

13.4.4 Triển khai các bộ lặp bất đồng bộ

Thay vì sử dụng trình tạo async để triển khai các trình lặp bất đồng bộ, cũng có thể triển khai chúng trực tiếp bằng cách định nghĩa một đối tượng có phương thức Symbol.asyncIterator(), trả về một đối tượng có phương thức next(), trả về một Promise, giải quyết thành một đối tượng kết quả trình lặp. Trong đoạn mã sau, chúng tôi tái triển khai hàm clock() từ ví dụ trước để nó không phải là một trình tạo, mà chỉ trả về một đối tượng có thể lặp bất đồng bộ. Lưu ý rằng phương thức next() trong ví dụ này không trả về một Promise một cách rõ ràng; thay vào đó, chúng tôi chỉ khai báo next() là async:

function clock(interval, max=Infinity) {
// A Promise-ified version of setTimeout that we can use await with.

This iterator-based version of the clock() function fixes a flaw in the generator-based version. Note that, in this newer code, we target the absolute time at which each iteration should begin and subtract the current time from that in order to compute the interval that we pass to setTimeout(). If we use clock() with a for/await loop, this version will run loop iterations more precisely at the specified in-terval because it accounts for the time required to actually run the body of the loop. But this fix isn’t just about timing accuracy. The for/await loop always waits for the Promise returned by one iteration to be fulfilled before it begins the next iteration. But if you use an asynchronous iterator without a for/await loop, there is nothing to prevent you from calling the next() method whenever you want. With the gen-erator-based version of clock(), if you call the next() method three times sequentially, you’ll get three Promises that will all fulfill at almost exactly the same time, which is probably not what you want. The it-erator-based version we’ve implemented here does not have that problem.

The benefit of asynchronous iterators is that they allow us to represent streams of asynchronous events or data. The clock() function discussed previously was fairly simple to write because the source of the asynchrony was the setTimeout() calls we were making ourselves. But when we are trying to work with other asynchronous sources, such as the triggering of event handlers, it becomes substantially harder to implement asynchronous iterators—we typically have a single event handler function that re-sponds to events, but each call to the iterator’s next() method must return a distinct Promise object, and multiple calls to next() may occur before the first Promise resolves. This means that any asynchronous iterator method must be able to maintain an internal queue of Promises that it resolves in order as asyn-chronous events are occurring. If we encapsulate this Promise-queueing behavior into an AsyncQueue class, then it becomes much easier to write asynchronous iterators based on AsyncQueue.

The AsyncQueue class that follows has enqueue() and dequeue() methods as you’d expect for a queue class. The dequeue() method returns a Promise rather than an actual value, however, which means that it is OK to call dequeue() before enqueue() has ever been called. The AsyncQueue class is also an asyn-chronous iterator, and is intended to be used with a for/await loop whose body runs once each time a new value is asynchronously enqueued. (AsyncQueue has a close() method. Once called, no more val-ues can be enqueued. When a closed queue is empty, the for/await loop will stop looping.)

Note that the implementation of AsyncQueue does not use async or await and instead works directly with Promises. The code is somewhat complicated, and you can use it to test your understanding of the material we’ve covered in this long chapter. Even if you don’t fully understand the AsyncQueue imple-mentation, do take a look at the shorter example that follows it: it implements a simple but very interest-ing asynchronous iterator on top of AsyncQueue.

Because this AsyncQueue class defines the asynchronous iteration basics, we can create our own, more interesting asynchronous iterators simply by asynchronously queueing values. Here’s an example that uses AsyncQueue to produce a stream of web browser events that can be handled with a for/await loop:

13.5 Summary

In this chapter, you have learned:

Most real-world JavaScript programming is asynchronous.

Traditionally, asynchrony has been handled with events and callback functions. This can get compli-cated, however, because you can end up with multiple levels of callbacks nested inside other callbacks, and because it is difficult to do robust error handling.

Promises provide a new way of structuring callback functions. If used correctly (and unfortunately, Promises are easy to use incorrectly), they can convert asynchronous code that would have been nested into linear chains of then() calls where one asynchronous step of a computation follows an-other. Also, Promises allow you to centralize your error-handling code into a single catch() call at the end of a chain of then() calls.

The async and await keywords allow us to write asynchronous code that is Promise-based under the hood but that looks like synchronous code. This makes the code easier to understand and reason about. If a function is declared async, it will implicitly return a Promise. Inside an async function, you can await a Promise (or a function that returns a Promise) as if the Promise value was synchronously computed.

Objects that are asynchronously iterable can be used with a for/await loop. You can create asyn-chronously iterable objects by implementing a [Symbol.asyncIterator]() method or by invoking an async function * generator function. Asynchronous iterators provide an alternative to “data” events on streams in Node and can be used to represent a stream of user input events in client-side JavaScript.

The XMLHttpRequest class has nothing in particular to do with XML. In modern client-side JavaScript, it has largely been replaced by the fetch() API, which is covered in §15.11.1. The code example shown here is the last XMLHttpRequest-based example remaining in this book.

You can typically use await at the top level in a browser’s developer console. And there is a pending proposal to al-low top-level await in a future version of JavaScript.

I learned about this approach to asynchronous iteration from the blog of Dr. Axel Rauschmayer, https://2ality.com.

// Note that this takes an absolute time instead of an interval.
function until(time) {
return new Promise(resolve => setTimeout(resolve, time - Date.now()));
}

// Return an asynchronously iterable object
return {
startTime: Date.now(), // Remember when we started
count: 1, // Remember which iteration we're on
async next() { // The next() method makes this an iterator
if (this.count > max) { // Are we done?
return { done: true }; // Iteration result indicating done
}
// Figure out when the next iteration should begin,
let targetTime = this.startTime + this.count * interval;
// wait until that time,
await until(targetTime);
// and return the count value in an iteration result object. return { value: this.count++ };
},
// This method means that this iterator object is also an iterable. [Symbol.asyncIterator]() { return this; }
};
}

Phiên bản dựa trên trình duyệt này của hàm clock() khắc phục một lỗi trong phiên bản dựa trên trình tạo. Lưu ý rằng, trong mã mới này, chúng tôi nhắm đến thời gian tuyệt đối mà mỗi lần lặp nên bắt đầu và trừ thời gian hiện tại để tính toán khoảng thời gian mà chúng tôi sẽ truyền cho setTimeout(). Nếu chúng ta sử dụng clock() với một vòng lặp for/await, phiên bản này sẽ thực hiện các lần lặp của vòng lặp một cách chính xác hơn, vì nó tính đến thời gian thực tế cần để thực hiện nội dung của vòng lặp. Tuy nhiên, sự khắc phục này không chỉ liên quan đến độ chính xác về thời gian. Với phiên bản dựa trên trình tạo của clock(), nếu bạn gọi phương thức next() ba lần liên tiếp,Bạn sẽ nhận được ba lời hứa; tất cả chúng đều sẽ được thực hiện gần như đồng thời. Có lẽ đó không phải là điều bạn mong muốn. Phiên bản dựa trên cơ chế lặp lại mà chúng tôi đã triển khai ởđây thì không gặp phải vấn đề này.

3

Ưu điểm của các trình duyệt không đồng bộ là chúng cho phép chúng ta biểu diễn luồng các sự kiện hoặc dữ liệu không đồng bộ. Hàm clock() mà chúng ta đã thảo luận trước đây khá đơn giản vì nguồn không đồng bộ nằm ở các cuộc gọi setTimeout(). Tuy nhiên, khi chúng ta cố gắng làm việc với các nguồn không đồng bộ khác, chẳng hạn như kích hoạt các hàm xử lý sự kiện, việc triển khai các trình duyệt không đồng bộ trở nên phức tạp hơn nhiều—thường thì chúng ta chỉ có một hàm xử lý sự kiện duy nhất, nhưng mỗi lần gọi đến phương thức next() của trình duyệt phải trả về một đối tượng Promise riêng biệt, và có thể có nhiều lần gọi next() trước khi đối tượng Promise đầu tiên được giải quyết. Điều này có nghĩa là bất kỳ phương thức trình duyệt không đồng bộ nào cũng phải có khả năng duy trì một hàng đợi bên trong các đối tượng Promise, giải quyết chúng theo thứ tự khi các sự kiện không đồng bộ xảy ra.Nếu chúng ta đóng gói chức năng xử lý các yêu cầu theo kiểu “Promise-queueing” này vào lớp AsyncQueue, thì việc tạo ra các công cụ xử lý dữ liệu một cách bất đồng bộ sẽ trở nên dễ dàng hơn nhiều.

3

Lớp AsyncQueue có các phương thức enqueue() và dequeue(), như mong đợi ở một lớp dùng để quản lý hàng đợi. Tuy nhiên, phương thức dequeue() trả về một giá trị dạng Promise thay vì giá trị thực tế. Điều này có nghĩa là chúng ta có thể gọi phương thức dequeue() trước khi gọi phương thức enqueue(). Lớp AsyncQueue cũng là một công cụ dùng để xử lý các thao tác một cách bất đồng bộ. Lớp này được thiết kếđể sử dụng cùng với vòng lặp for/await: vòng lặp này sẽ được thực hiện mỗi lần có giá trị mới được thêm vào hàng đợi một cách bất đồng bộ. Lớp AsyncQueue còn có phương thức close(). Khi phương thức này được gọi, sẽ không còn giá trị nào có thểđược thêm vào hàng đợi nữa. Khi hàng đợi đã trống, vòng lặp for/await sẽ ngừng hoạt động.

Lưu ý rằng cách triển khai của AsyncQueue không sử dụng các ký hiệu async hay await. Thay vào đó, AsyncQueue hoạt động trực tiếp với các đối tượng Promises. Đoạn mã này khá phức tạp; bạn có thể sử dụng nó để kiểm tra mức độ hiểu biết của mình về những nội dung đã được trình bày trong chương này. Ngay cả khi bạn chưa hiểu hết cách triển khai của AsyncQueue, hãy thử xem ví dụ đơn giản hơn ở phía dưới: ví dụ này minh họa cách tạo ra một bộ lặp phi tuyến đơn giản nhưng rất hữu ích, dựa trên cơ sở của AsyncQueue.

/**
* An asynchronously iterable queue class. Add values with enqueue()
* and remove them with dequeue(). dequeue() returns a Promise, which
* means that values can be dequeued before they are enqueued. The
* class implements [Symbol.asyncIterator] and next() so that it can
* be used with the for/await loop (which will not terminate until
* the close() method is called.)
*/
class AsyncQueue {
constructor() {
// Values that have been queued but not dequeued yet are stored here this.values = [];
// When Promises are dequeued before their corresponding values are
// queued, the resolve methods for those Promises are stored here. this.resolvers = [];
// Once closed, no more values can be enqueued, and no more unfulfilled
// Promises returned.
this.closed = false;
}

enqueue(value) {
if (this.closed) {
throw new Error("AsyncQueue closed");
}
if (this.resolvers.length > 0) {
// If this value has already been promised, resolve that Promise const resolve = this.resolvers.shift();
resolve(value);
}
else {
// Otherwise, queue it up
this.values.push(value);
}
}

dequeue() {
if (this.values.length > 0) {
// If there is a queued value, return a resolved Promise for it const value = this.values.shift();
return Promise.resolve(value);
}
else if (this.closed) {
// If no queued values and we're closed, return a resolved
// Promise for the "end-of-stream" marker
return Promise.resolve(AsyncQueue.EOS);
}
else {
// Otherwise, return an unresolved Promise,
// queuing the resolver function for later use
return new Promise((resolve) => { this.resolvers.push(resolve); });
}
}

close() {
// Once the queue is closed, no more values will be enqueued.
// So resolve any pending Promises with the end-of-stream marker while(this.resolvers.length > 0) { this.resolvers.shift()(AsyncQueue.EOS);
}
this.closed = true;
}

// Define the method that makes this class asynchronously iterable [Symbol.asyncIterator]() { return this; }

// Define the method that makes this an asynchronous iterator. The // dequeue() Promise resolves to a value or the EOS sentinel if we're // closed. Here, we need to return a Promise that resolves to an
// iterator result object.
next() {
return this.dequeue().then(value => (value === AsyncQueue.EOS)
? { value: undefined, done: true }
: { value: value, done: false });
}
}

// A sentinel value returned by dequeue() to mark "end of stream" when closed AsyncQueue.EOS = Symbol("end-of-stream");

Vì lớp AsyncQueue định nghĩa các nguyên lý cơ bản của việc xử lý dữ liệu một cách bất đồng bộ, chúng ta có thể tạo ra những bộ lặp bất đồng bộ riêng của mình một cách dễ dàng, chỉ cần xếp các giá trị vào hàng đợi một cách bất đồng bộ. Dưới đây là ví dụ về cách sử dụng AsyncQueue để tạo ra dòng sự kiện từ trình duyệt web; những sự kiện này có thểđược xử lý bằng vòng lặp for/await.

// Push events of the specified type on the specified document element
// onto an AsyncQueue object, and return the queue for use as an event stream
function eventStream(elt, type) {
const q = new AsyncQueue(); // Create a queue elt.addEventListener(type, e=>q.enqueue(e)); // Enqueue events
return q;
}

async function handleKeys() {
// Get a stream of keypress events and loop once for each one
for await (const event of eventStream(document, "keypress")) {
console.log(event.key);
}
}

13.5 Tóm tắt Trong chương này, bạn đã học được những điều sau:
Hầu hết các chương trình lập trình bằng JavaScript trong thế giới thực đều được thực hiện theo kiểu bất đồng bộ.

Theo truyền thống, việc xử lý các tình huống không đồng bộ thường được thực hiện thông qua các sự kiện và hàm gọi lại. Tuy nhiên, cách tiếp cận này có thể gây ra nhiều rắc rối: có thể xuất hiện tình trạng nhiều lớp hàm gọi lại chồng chất lên nhau; đồng thời, việc xử lý các lỗi cũng trở nên khó khăn.

Promises mang lại cách tiếp cận mới để tổ chức các hàm gọi lại một cách hiệu quả. Nếu được sử dụng đúng cách (mặc dù thật không may, Promises rất dễ bị sử dụng sai cách), chúng có thể giúp chuyển đổi các đoạn mã phi tuyến thành các chuỗi các lệnh gọi tuần tự. Ngoài ra, Promises còn cho phép tập trung việc xử lý các lỗi vào một lệnh gọi duy nhất, ở cuối chuỗi các lệnh gọi khác.

Các từ khóa async và await giúp chúng ta viết mã theo kiểu bất đồng bộ. Về mặt cơ bản, đây là mã dựa trên cấu trúc Promise, nhưng về mặt hình thức thì trông giống như mã đồng bộ. Điều này giúp mã dễ hiểu và dễ sử dụng hơn. Nếu một hàm được đánh dấu là async, thì hàm đó sẽ trả về một giá trị kiểu Promise một cách ngầm định. Bên trong một hàm được đánh dấu là async, chúng ta có thể sử dụng giá trị của Promise đó như thể giá trị đó được tính toán một cách đồng bộ vậy.

Các đối tượng có khả năng được duyệt một cách bất đồng bộ có thểđược sử dụng trong vòng lặp
for/await. Bạn có thể tạo ra các đối tượng như vậy bằng cách triển khai phương thức [Symbol.asyncIterator]() hoặc gọi hàm tạo ra dữ liệu kiểu async function *

. Các bộ lặp bất đồng bộ
này là giải pháp thay thế cho các sự kiện “dữ liệu” trong các luồng dữ liệu của Node.js. Chúng cũng có thể
được sử dụng để biểu diễn các sự kiện đầu vào từ người dùng trong JavaScript ở phía máy khách.

1

Lớp XMLHttpRequest thực chất không có mối liên hệ gì đặc biệt với định dạng XML cả. Trong các chương trình JavaScript hoạt động ở phía máy khách ngày nay, lớp này đã bị thay thế phần lớn bởi API fetch(), vốn được trình bày chi tiết trong mục §15.11.1. Ví dụ mã nguồn được trình bày ởđây là ví dụ cuối cùng sử dụng lớp XMLHttpRequest trong cuốn sách này.

1

2

Thông thường, bạn có thể sử dụng await ở cấp độ cao nhất trong bảng điều khiển phát triển của trình duyệt. Ngoài ra, hiện có đề xuất để cho phép việc sử dụng await ở cấp độ cao nhất trong các phiên bản JavaScript tương lai.

2

3

3 Tôi biết về phương pháp xử lý các lệnh lặp bất đồng bộ này từ blog của Tiến sĩ Axel Rauschmayer: https://2ality.com.

.